

Inference Providers · Hugging Face

Source: <https://huggingface.co/zai-org/GLM-4.7-Flash>

[Hugging Face](#)

[Models](#)

[Datasets](#)

[Spaces](#)

[Buckets](#)

[NEW](#)

[Docs](#)

[Enterprise](#)

[Pricing](#)

[Log In](#)

[Sign Up](#)

[Inference Providers](#)

 [View all docs](#)

[AWS Trainium & Inferentia](#)

[Accelerate](#)

[Argilla](#)

[AutoTrain](#)

[Bitsandbytes](#)

[CLI](#)

[Chat UI](#)

[Dataset viewer](#)

[Datasets](#)

[Deploying on AWS](#)

[Diffusers](#)

[Distilabel](#)

[Evaluate](#)

[Google Cloud](#)

[Google TPUs](#)

[Gradio](#)

[Hub](#)

[Hub Python Library](#)

[Huggingface.js](#)

[Inference Endpoints \(dedicated\)](#)

[Inference Providers](#)

[Kernels](#)

[LeRobot](#)

[Leaderboards](#)

[Lighteval](#)

[Microsoft Azure](#)

[Optimum](#)

[PEFT](#)

[Reachy Mini](#)

[Safetensors](#)

[Sentence Transformers](#)

[TRL](#)

[Tasks](#)

[Text Embeddings Inference](#)

[Text Generation Inference](#)

[Tokenizers](#)

Trackio
Transformers
Transformers.js
Xet
smolagents
timm
Search documentation
%K
main
EN
GET STARTED
Inference Providers
Pricing and Billing
Hub integration
Security
GUIDES
Your First API Call
Building Your First AI App
Structured Outputs with LLMs
Function Calling
Responses API (beta)
How to use OpenAI gpt-oss
Build an Image Editor
Automating Code Review with GitHub Actions
Agentic Coding Environments with OpenEnv
Evaluating Models with Inspect
INTEGRATIONS
Overview
Add Your Integration
Hermes Agent
NeMo Data Designer
MacWhisper
OpenCode
Pi
Vision Agents
VS Code with GitHub Copilot
INFERENCE TASKS
Chat Completion
Feature Extraction
Text to Image
Text to Video
OTHER TASKS
PROVIDERS
Cerebras
Cohere
Fal AI
Featherless AI
Fireworks
Groq
Hyperbolic
HF Inference
Novita
Nscale
OVHcloud AI Endpoints
Public AI

[Replicate](#)
[SambaNova](#)
[Scaleway](#)
[Together](#)
[WaveSpeedAI](#)
[Z.ai](#)
[Hub API](#)
[Register as an Inference Provider](#)
[Join the Hugging Face community](#)

and get access to the augmented documentation experience

[Collaborate on models, datasets and Spaces](#)
[Faster examples with accelerated inference](#)
[Switch between documentation themes](#)
[Sign Up](#)

to get started

[Copy page](#)
[Inference Providers](#)

Hugging Face's Inference Providers give developers access to hundreds of machine learning models, powered by world-class inference providers. They are also integrated into our client SDKs (for JS and Python), making it easy to explore serverless inference of models on your favorite providers.

Partners

Our platform integrates with leading AI infrastructure providers, giving you access to their specialized capabilities through a single, consistent API. Here's what each partner supports:

Provider	Chat completion (LLM)		Chat completion (VLM)		Feature Extraction	Text
	Text to video	Speech to text				
Cerebras	✓					
Cohere ✓	✓					
Fal AI		✓	✓	✓		
Featherless AI	✓	✓				
Fireworks	✓	✓				
Groq ✓	✓					
HF Inference	✓	✓	✓	✓		✓
Hyperbolic	✓	✓				
Novita ✓	✓			✓		
Nscale ✓	✓		✓			
OVHcloud AI Endpoints		✓	✓			
Public AI	✓					
Replicate				✓	✓	✓
SambaNova	✓		✓			
Scaleway	✓		✓			
Together	✓	✓		✓		
WaveSpeedAI				✓	✓	
Z.ai ✓	✓					

[Why Choose Inference Providers?](#)

When you build AI applications, it's tough to manage multiple provider APIs, comparing model performance, and dealing with varying reliability. Inference Providers solves these challenges by offering:

Instant Access to Cutting-Edge Models: Go beyond mainstream providers to access thousands of specialized models across multiple AI tasks. Whether you need the latest language models, state-of-the-art image generators, or domain-specific embeddings, you'll find them here.

Zero Vendor Lock-in: Unlike being tied to a single provider's model catalog, you get access to models from Cerebras, Groq, Together AI, Replicate, and more — all through one consistent interface.

Production-Ready Performance: Built for enterprise workloads with the reliability your applications demand.

Here's what you can build:

Text Generation: Use Large language models with tool-calling capabilities for chatbots, content generation, and code assistance

Image and Video Generation: Create custom images and videos, including support for LoRAs and style customization

Search & Retrieval: State-of-the-art embeddings for semantic search, RAG systems, and recommendation engines

Traditional ML Tasks: Ready-to-use models for classification, NER, summarization, and speech recognition

⚡ **Get Started for Free:** Inference Providers includes a generous free tier, with additional credits for PRO users and Team & Enterprise organizations.

Key Features

🌀 **All-in-One API:** A single API for text generation, image generation, document embeddings, NER, summarization, image classification, and more.

🔄 **Multi-Provider Support:** Easily run models from top-tier providers like fal, Replicate, Sambanova, Together AI, and others.

🚀 **Scalable & Reliable:** Built for high availability and low-latency performance in production environments.

🔧 **Developer-Friendly:** Simple requests, fast responses, and a consistent developer experience across Python and JavaScript clients.

👤 **Easy to integrate:** Drop-in replacement for the OpenAI chat completions API.

💰 **Cost-Effective:** No extra markup on provider rates.

Getting Started

Inference Providers works with your existing development workflow. Whether you prefer Python, JavaScript, or direct HTTP calls, we provide native SDKs and OpenAI-compatible APIs to get you up and running quickly.

We'll walk through a practical example using openai/gpt-oss-120b, a state-of-the-art open-weights conversational model.

Inference Playground

Before diving into integration, explore models interactively with our Inference Playground. Test different chat completion models with your prompts and compare responses to find the perfect fit for your use case.

Authentication

You'll need a Hugging Face token to authenticate your requests. Create one by visiting your token settings and generating a fine-grained token with Make calls to Inference Providers permissions.

For complete token management details, see our security tokens guide.

Quick Start - LLM

Let's start with the most common use case: conversational AI using large language models. This section demonstrates how to perform chat completions using DeepSeek V3, showcasing the different ways you can integrate Inference Providers into your applications.

Whether you prefer our native clients, want OpenAI compatibility, or need direct HTTP access, we'll show you how to get up and running with just a few lines of code.

Python

Here are three ways to integrate Inference Providers into your Python applications, from high-level convenience to low-level control:

```
huggingface_hub
openai
requests
```

For convenience, the `huggingface_hub` library provides an `InferenceClient` that automatically handles provider selection and request routing.

In your terminal, install the Hugging Face Hub Python client and log in:

```
Copied
pip install huggingface_hub
hf auth login # get a read token from hf.co/settings/tokens
```

You can now use the client with a Python interpreter.

By default, our system automatically selects the fastest available provider for the specified model (equivalent to the `:fastest` policy – highest throughput in tokens per second).

You can change the provider selection policy by appending a policy suffix to the model id: `:cheapest` for the most cost-efficient provider (lowest price per output token), or `:preferred` to follow your preference order in Inference Provider settings. For example, `openai/gpt-oss-120b:cheapest`.

You can also select the provider of your choice by appending the provider name to the model id (e.g. `"openai/gpt-oss-120b:sambanova"`).

```
Copied
import os
from huggingface_hub import InferenceClient

client = InferenceClient()
```

```
completion = client.chat.completions.create(
  model="openai/gpt-oss-120b",
  messages=[
    {
      "role": "user",
      "content": "How many 'G's in 'huggingface'?"
    }
  ],
)
```

```
print(completion.choices[0].message)
JavaScript
```

Integrate Inference Providers into your JavaScript applications with these flexible approaches:

```
huggingface.js
openai
fetch
```

Our JavaScript SDK provides a convenient interface with automatic provider selection and TypeScript support.

Install with NPM:

```
Copied
npm install @huggingface/inference
```

Then use the client with Javascript.

By default, our system automatically selects the fastest available provider for the specified model (equivalent to the `:fastest` policy – highest throughput in tokens per second).

You can change the provider selection policy by appending a policy suffix to the model id: `:cheapest` for the most cost-efficient provider (lowest price per output token), or `:preferred` to follow your preference order in Inference Provider settings. For example, `openai/gpt-oss-120b:cheapest`.

You can also select the provider of your choice by appending the provider name to the model id (e.g. `"openai/gpt-oss-120b:sambanova"`).

```
Copied
import { InferenceClient } from "@huggingface/inference";
```

```
const client = new InferenceClient(process.env.HF_TOKEN);
```

```
const chatCompletion = await client.chatCompletion({
  model: "openai/gpt-oss-120b:fastest",
  messages: [
    {
      role: "user",
      content: "How many 'G's in 'huggingface'?",
    },
  ],
},
```

```
});
```

```
console.log(chatCompletion.choices[0].message);  
HTTP / cURL
```

For testing, debugging, or integrating with any HTTP client, here's the raw REST API format.

By default, our system automatically selects the fastest available provider for the specified model (equivalent to the `:fastest` policy – highest throughput in tokens per second).

You can change the provider selection policy by appending a policy suffix to the model id: `:cheapest` for the most cost-efficient provider (lowest price per output token), or `:preferred` to follow your preference order in Inference Provider settings. For example, `openai/gpt-oss-120b:cheapest`.

You can also select the provider of your choice by appending the provider name to the model id (e.g. `"openai/gpt-oss-120b:sambanova"`).

Copied

```
curl https://router.huggingface.co/v1/chat/completions \  
  -H "Authorization: Bearer $HF_TOKEN" \  
  -H 'Content-Type: application/json' \  
  -d '{  
    "messages": [  
      {  
        "role": "user",  
        "content": "How many G in huggingface?"  
      }  
    ],  
    "model": "openai/gpt-oss-120b:fastest",  
    "stream": false  
  }'
```

Quick Start - Text-to-Image Generation

Let's explore how to generate images from text prompts using Inference Providers. We'll use `black-forest-labs/FLUX.1-dev`, a state-of-the-art diffusion model that produces highly detailed, photorealistic images.

Python

Use the `huggingface_hub` library for the simplest image generation experience with automatic provider selection:

Copied

```
import os  
from huggingface_hub import InferenceClient  
  
client = InferenceClient(api_key=os.environ["HF_TOKEN"])  
  
image = client.text_to_image(  
    prompt="A serene lake surrounded by mountains at sunset, photorealistic style",  
    model="black-forest-labs/FLUX.1-dev"  
)
```

```
# Save the generated image
image.save("generated_image.png")
JavaScript
```

Use our JavaScript SDK for streamlined image generation with TypeScript support:

```
Copied
import { InferenceClient } from "@huggingface/inference";
import fs from "fs";

const client = new InferenceClient(process.env.HF_TOKEN);

const imageBlob = await client.textToImage({
  model: "black-forest-labs/FLUX.1-dev",
  inputs:
    "A serene lake surrounded by mountains at sunset, photorealistic style",
});

// Save the image
const buffer = Buffer.from(await imageBlob.arrayBuffer());
fs.writeFileSync("generated_image.png", buffer);
Provider Selection
```

The Inference Providers API acts as a unified proxy layer that sits between your application and multiple AI providers. Understanding how provider selection works is crucial for optimizing performance, cost, and reliability in your applications.

API as a Proxy Service

When using Inference Providers, your requests go through Hugging Face's proxy infrastructure, which provides several key benefits:

Unified Authentication & Billing: Use a single Hugging Face token for all providers

Automatic Failover: When using automatic provider selection (`provider="auto"`), requests are automatically routed to alternative providers if the primary provider is flagged as unavailable by our validation system

Consistent Interface through client libraries: When using our client libraries, the same request format works across different providers

Because the API acts as a proxy, the exact HTTP request may vary between providers as each provider has their own API requirements and response formats. When using our official client libraries (JavaScript or Python), these provider-specific differences are handled automatically whether you use `provider="auto"` or specify a particular provider.

Client-Side Provider Selection (Inference Clients)

When using the Hugging Face inference clients (JavaScript or Python), you can explicitly specify a provider or let the system choose automatically. The client then formats the HTTP request to match the selected provider's API requirements.

```
javascript
python
Copied
import { InferenceClient } from "@huggingface/inference";
```



```
const client = new InferenceClient(process.env.HF_TOKEN);
```

```
// Explicit provider selection
await client.chatCompletion({
  model: "deepseek-ai/DeepSeek-R1",
  provider: "sambanova", // Specific provider
  messages: [{ role: "user", content: "Hello!" }],
});
```

```
// Automatic provider selection (default: "auto")
await client.chatCompletion({
  model: "deepseek-ai/DeepSeek-R1",
  // Defaults to "auto" selection of the provider
  // provider="auto",
  messages: [{ role: "user", content: "Hello!" }],
});
```

Provider Selection Policy:

provider: "auto" (default): Selects the first available provider for the model, sorted by your preference order in Inference Provider settings.

provider: "specific-provider": Forces use of a specific provider (e.g., "together", "replicate", "fal-ai", ...).

Alternative: OpenAI-Compatible Chat Completions Endpoint (Chat Only)

If you prefer to work with familiar OpenAI APIs or want to migrate existing chat completion code with minimal changes, we offer a drop-in compatible endpoint that handles all provider selection automatically on the server side.

By default, the fastest available provider is selected for the model (highest throughput in tokens per second). This is equivalent to appending :fastest to the model name. You can change that policy by adding a suffix to the model name:

:cheapest selects the most cost-efficient provider for the model (lowest price per output tokens)

:preferred selects the first available provider sorted by your preference order in Inference Provider settings

Note: This OpenAI-compatible endpoint is currently available for chat completion tasks only. For other tasks like text-to-image, embeddings, or speech processing, use the Hugging Face inference clients shown above.

javascript

python

Copied

```
import { OpenAI } from "openai";
```

```
const client = new OpenAI({
  baseURL: "https://router.huggingface.co/v1",
  apiKey: process.env.HF_TOKEN,
});
```

```
const completion = await client.chat.completions.create({
  model: "deepseek-ai/DeepSeek-R1:fastest",
  messages: [{ role: "user", content: "Hello!" }],
});
```

```
});
```

This endpoint can also be requested through direct HTTP access, making it suitable for integration with various HTTP clients and applications that need to interact with the chat completion service directly.

Copied

```
curl https://router.huggingface.co/v1/chat/completions \  
-H "Authorization: Bearer $HF_TOKEN" \  
-H "Content-Type: application/json" \  
-d '{  
  "model": "deepseek-ai/DeepSeek-R1:fastest",  
  "messages": [  
    {  
      "role": "user",  
      "content": "Hello!"  
    }  
  ]  
}'
```

Key Features:

Server-Side Provider Selection: The server automatically selects the fastest available provider by default (:fastest policy)

Model Listing: GET /v1/models returns available models across all providers

OpenAI SDK Compatibility: Works with existing OpenAI client libraries

Chat Tasks Only: Limited to conversational workloads

Choosing the Right Approach

Use Inference Clients when:

You need support for all task types (text-to-image, speech, embeddings, etc.)

You want explicit control over provider selection

You're building applications that use multiple AI tasks

Use OpenAI-Compatible Endpoint when:

You're only doing chat completions

You want to migrate existing OpenAI-based code with minimal changes

You prefer server-side provider management

Use Direct HTTP when:

You're implementing custom request logic

You need fine-grained control over the request/response cycle

You're working in environments without available client libraries

Next Steps

Now that you understand the basics, explore these resources to make the most of Inference Providers:

Announcement Blog Post: Learn more about the launch of Inference Providers

Pricing and Billing: Understand costs and billing of Inference Providers

Hub Integration: Learn how Inference Providers are integrated with the Hugging Face Hub

Register as a Provider: Requirements to join our partner network as a provider

Hub API: Advanced API features and configuration

API Reference: Complete parameter documentation for all supported tasks

Update on GitHub

Pricing and Billing

→

Inference Providers

Partners

Why Choose Inference Providers?

Key Features

Getting Started

Inference Playground

Authentication

Quick Start - LLM

Python

JavaScript

HTTP / cURL

Quick Start - Text-to-Image Generation

Python

JavaScript

Provider Selection

API as a Proxy Service

Client-Side Provider Selection (Inference Clients)

Alternative: OpenAI-Compatible Chat Completions Endpoint (Chat Only)

Choosing the Right Approach

Next Steps